T.C.

KOCAELİ UNIVERSITY

FACULTY OF ENGINEERING

MECHANICAL ENGINEERING DEPARTMENT

SOLUTION OF 1D HEAT EQUATION WITH NUMERICAL METHODS

Thermal Design Project

Yusuf Ziya ÇAKMAK

170218082

Advisor: Arş. Gör. Dr. Özgür KAPLAN

JUNE 2020

SUMMARY

In this study, analytical and numerical solutions of 1D heat conduction equation were compared. During numerical operations, python and NumPy library are used. For the sample problem, the mentioned methods were applied and RMSE values were calculated. For the sample problem solved by looking at the RMSE values, the method with the smallest error value is the explitic method.

Keywords: Heat Transfer, Numerical Methods, Explitic Method, Implitic Method, Crank-Nicolson Method,

CONTENTS

ABBREVIATIONS

u_i^k: temperature at t=k and x=i

RMSE: Root Mean Square Error

SHAPES INDEX

TABLES INDEX

INTRODUCTION

The objective of this study is to determine among the numerical methods in the study, will produce the closest results to the analytical solution. This study starts with the solution of one dimensional heat conduction equation analytically under given boundary conditions. Then, the solution of the same equation is given with different numerical methods. Then, for the sample problem, the temperature profile at a certain time was created using the methods mentioned in the study. Error rates were found by comparing the calculated temperature profiles with the temperature profile of the analytical solution.

1. ANALYTIC PART[1]

1D heat conduction equation.

$$\frac{\partial u}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2} \tag{1.1}$$

$$c^2 = \alpha(thermal\ diffusivity)$$

$$BCs\ and\ IC$$

$$u(0,t) = 0$$

$$u(L,t) = 0$$

$$u(x,0) = f(x)$$

Function is seperated to two distinct one variable functions.

$$u(x,t) = F(x)G(t)$$

Thus, their derivatives are fully derived.

$$\frac{\partial u}{\partial t} = F\dot{G} \qquad \frac{\partial^2 u}{\partial x^2} = F''G$$

Derivatives are plug into the Eq. (1.1)

$$F\dot{G} = c^2 F''G$$

$$\frac{\dot{G}}{c^2 G} = \frac{F''}{F}$$

$$\frac{\dot{G}}{c^2 G} = \frac{F''}{F} = -p^2$$

Consequentially, two ordinary differential equations are obtained

$$F'' + p^2 F = 0 \tag{1.2}$$

$$\dot{G} + c^2 p^2 G = 0 \tag{1.3}$$

First solve the Eq. (1.2) a general solution is

$$F(x) = Acos(px) + Bsin(px)$$

$$u(0,t) = F(0)G(t) = 0$$

$$u(L,t) = F(L)G(t) = 0$$

Since $G(t) = 0$ would give $u = 0$, we require $F(0) = 0, F(L) = 0$ and get $F(0) = A = 0$ by (3) and then $F(L) = Bsin(pL) = 0$ with $B \neq 0$ thus,

$\sin(pL) = 0,$ hence $p = \frac{n\pi}{L}, \quad n = 1,2,\cdots.$

$$F_n(x) = \sin\left(\frac{n\pi x}{L}\right), \quad n = 1,2,\cdots.$$

We now solve Eq. (3) For $p = \frac{n\pi}{L}$, Eq. (3) becomes,

$$\dot{G} + \lambda_n^2 G = 0 \quad where \quad \lambda_n = \frac{cn\pi}{L}$$

It has general solution,

$$G_n = B_n e^{-\lambda_n^2 t}$$

Where $B_n$is constant hence the functions

$$u_n(x,t) = F_n(x)G_n(t) = B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\lambda_n^2 t}$$

Solution of the entire problem.Fourier series.To obtain a solution that also satisfies the initial condition

$$u(x,t) = \sum_{n=1}^{\infty} u_n(x,t) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) e^{-\lambda_n^2 t} \qquad (1.4)$$

From Eq. (1.4) we have,

$$u(x,0) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) = f(x)$$

The Bn must be the coefficents of the Fourier sine series as given by,

$$B_n = \frac{2}{L}\int_0^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx$$

Thus exact solution is

$$u(x,t) = \sum_{n=1}^{\infty} \left[\frac{2}{L}\int_0^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx\right] \sin\left(\frac{n\pi x}{L}\right) e^{-\lambda_n^2 t}$$

2. NUMERICAL PART

Finite difference Approximations[2]

First order forward difference

Use a Taylor series expansion about the point x_i

$$u(x_i + \delta x) = u(x_i) + \delta x \frac{\partial u}{\partial x}\Big|_{x_i} + \frac{\delta x^2}{2!}\frac{\partial^2 u}{\partial x^2}\Big|_{x_i} + \frac{\delta x^3}{3!}\frac{\partial^3 u}{\partial x^3}\Big|_{x_i} + \cdots$$

where δx is a small distance, and δx^2 and δx^3 are shorthand for $(\delta x)^2$ and $(\delta x)^3$.

Solve for $\frac{\partial u}{\partial x}\Big|_{x_i}$

$$\frac{\partial u}{\partial x}\Big|_{x_i} = \frac{u(x_i, \delta x) - u(x_i)}{\delta x} - \frac{\delta x}{2!}\frac{\partial^2 u}{\partial x^2}\Big|_{x_i} - \frac{\delta x^2}{3!}\frac{\partial^3 u}{\partial x^3}\Big|_{x_i} + \cdots$$

In the limit as $\delta x \to 0$ the coefficients of the higher order derivative terms vanish, and the first term on the right hand side becomes equal to the derivative on the left hand side.

Replace δx with $\Delta x = x_{i+1} - x_i$, and substitute $u_i \approx u(x_i)$

$$\frac{\partial u}{\partial x}\Big|_{x_i} \approx \frac{u_{i+1} - u_i}{\Delta x} - \frac{\Delta x}{2!}\frac{\partial^2 u}{\partial x^2}\Big|_{x_i} - \frac{(\Delta x)^2}{3!}\frac{\partial^3 u}{\partial x^3}\Big|_{x_i} + \cdots$$

SinceΔx is the only parameter under the user's control that determines the error, the truncation error is simply written

$$\frac{\Delta x}{2!}\frac{\partial^2 u}{\partial x^2}\Big|_{\xi} = O(\Delta \mathrm{x})$$

Recall that the point of Big O notation is to look at the exponent of the truncation error term. We conclude that the error in the first order forward difference formula is linear in the mesh spacing.

$$\frac{\partial u}{\partial x}\Big|_{x_i} = \frac{u_{i+1} - u_i}{\Delta x} + O(\Delta \mathrm{x})$$

First order backward difference

Repeat the preceding analysis with a Taylor series expansion for $u(x_i - \delta x)$.

$$\left.\frac{\partial u}{\partial x}\right|_{x_i} = \frac{u_i - u_{i-1}}{\Delta x} - \frac{\Delta x}{2!}\left.\frac{\partial^2 u}{\partial x^2}\right|_{x_i} - \frac{(\Delta x)^2}{3!}\left.\frac{\partial^3 u}{\partial x^3}\right|_{x_i} + \cdots$$

or

$$\left.\frac{\partial u}{\partial x}\right|_{x_i} = \frac{u_i - u_{i-1}}{\Delta x} + O(\Delta \mathrm{x})$$

First order central difference

Combine Taylor Series expansions for u_{i+1} and u_{i-1} to get

$$\left.\frac{\partial u}{\partial x}\right|_{x_i} = \frac{u_{i+1} - u_{i-1}}{2\Delta x} + O(\Delta \mathrm{x}^2)$$

Second order central difference

$$\left.\frac{\partial^2 u}{\partial x^2}\right|_{x_i} = \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2} + O(\Delta \mathrm{x}^2)$$

Summary of finite difference approximations

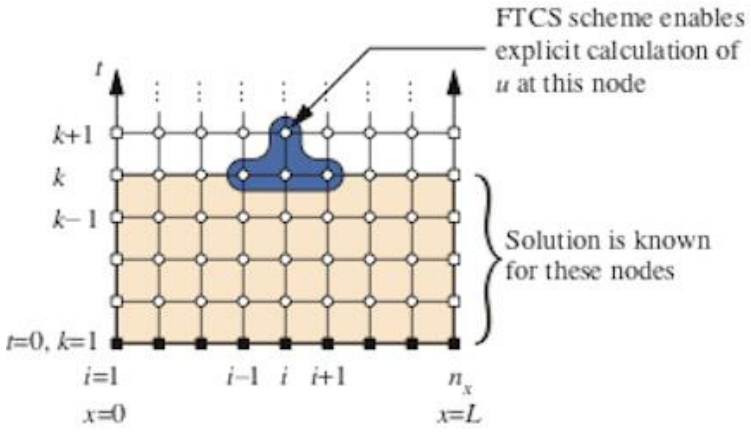
Forward: $\left.\frac{\partial u}{\partial x}\right|_{x_i} = \frac{u_{i+1} - u_i}{\Delta x} + O(\Delta \mathrm{x})$

Backward: $\left.\frac{\partial u}{\partial x}\right|_{x_i} = \frac{u_i - u_{i-1}}{\Delta x} + O(\Delta \mathrm{x})$

Central: $\left.\frac{\partial u}{\partial x}\right|_{x_i} = \frac{u_{i+1} - u_{i-1}}{2\Delta x} + O(\Delta \mathrm{x}^2)$

Second Order Central: $\left.\frac{\partial^2 u}{\partial x^2}\right|_{x_i} = \frac{u_{i-1} - 2u_i + u_{i+1}}{\Delta x^2} + O(\Delta \mathrm{x}^2)$

2.1 Explicit Method



[3]

Shape 2.1.1

We are applying discretization to 1D heat conduction equation.

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2} \tag{2.1.1}$$

$\partial u/\partial t$ at $x = x_i$

$$\frac{\partial u}{\partial t} = \frac{u_{x_i}^{k+1} - u_{x_i}^{k}}{\Delta t}$$

$\partial^2 u/\partial x^2$ at $t = k$

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k}}{\Delta x^2}$$

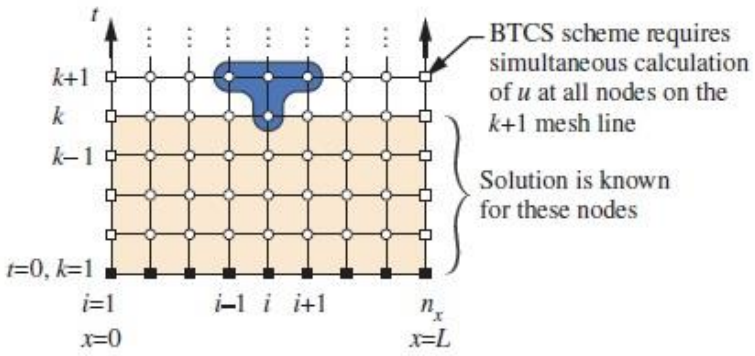
Eq.(2.1.1) becomes

$$\frac{u_{x_i}^{k+1} - u_{x_i}^{k}}{\Delta t} = \alpha \frac{u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k}}{\Delta x^2} \tag{2.1.2}$$

We assume that $fou = \alpha \frac{\Delta t}{\Delta x^2}$ and organize Eq. (2)

$$u_{x_i}^{k+1} = u_{x_i}^{k} + fou\left(u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k}\right)$$

2.2 Implitic Method



[4]

Shape 2.2.1

We are applying discretization to general 1D heat conduction equation.

We already discretizated the equation for the Explitic Method and we obtained Eq(2.1.2).

$$\frac{u_{x_i}^{k+1} - u_{x_i}^{k}}{\Delta t} = \alpha \frac{u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k}}{\Delta x^2}$$

Shift the k subscripts by one, and move all u^{k+1} terms to the left hand side. We assume that $fou = \alpha \frac{\Delta t}{\Delta x^2}$

Eq.(2.1.2) becomes,

$$[-fou]u_{x_{i-1}}^{k+1} + [1 + 2fou]u_{x_i}^{k+1} + [-fou]u_{x_{i+1}}^{k+1} = u_{x_i}^{k} \qquad (2.2.1)$$

Convert to Eq.(2.2.1) to matrix form. N is number of points.

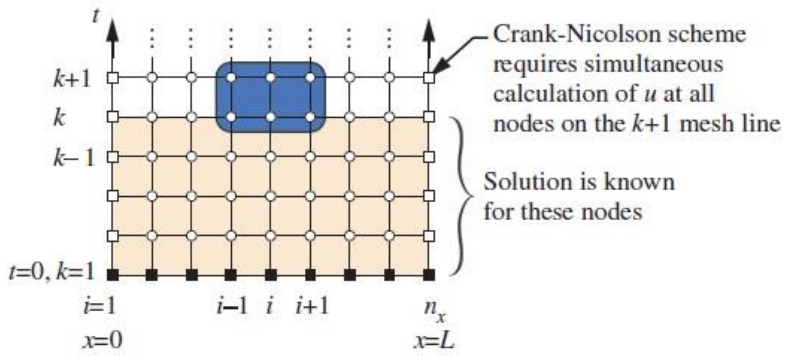
$$[A]\{x\} = [B]$$

$$[A] = \begin{bmatrix} 1+2fou & -fou & 0 & 0 & \dots & 0 \\ 0 & -fou & 1+2fou & -fou & \dots & 0 \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot & \cdot \\ 0 & 0 & 0 & \dots & -2fou & 1+2fou \end{bmatrix} \Big\} N-2$$

$$\{x\} = \begin{Bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{Bmatrix}$$

$$[B] = \begin{bmatrix} u_1^k \\ u_2^k \\ \vdots \\ u_n^k \end{bmatrix}$$

2.3 Cranck-Nicolson Method



[5]

Shape 2.3.1

We are applying discretization to 1D heat conduction equation.

Average of the $\partial^2 u/\partial x^2$ operators at time t_k and t_{k+1}

$$\frac{\partial^2 u}{\partial x^2} = \frac{1}{2}\left[\frac{u_{x_{i-1}}^{k+1} - 2u_{x_i}^{k+1} + u_{x_{i+1}}^{k+1}}{\Delta x^2}\right] + \frac{1}{2}\left[\frac{u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k}}{\Delta x^2}\right]$$

$$\frac{u_{x_i}^{k+1} - u_{x_i}^{k}}{\Delta t} = \frac{\alpha}{2}\left[\frac{u_{x_{i-1}}^{k+1} - 2u_{x_i}^{k+1} + u_{x_{i+1}}^{k+1}}{\Delta x^2}\right] + \frac{\alpha}{2}\left[\frac{u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k}}{\Delta x^2}\right]$$

And move all u^{k+1} terms to the left hand side and if we assume that $fou = \alpha\frac{\Delta t}{\Delta x^2}$

$$[-fou]u_{x_{i-1}}^{k+1} + [2 + 2fou]u_{x_i}^{k+1} + [-fou]u_{x_{i+1}}^{k+1} = 2u_{x_i}^{k} + fou(u_{x_{i-1}}^{k} - 2u_{x_i}^{k} + u_{x_{i+1}}^{k})$$

Convert to this equation to matrix form. N is number of points.

$$[A] = \begin{bmatrix} 2+2fou & -fou & 0 & 0 & \dots & 0 \\ 0 & -fou & 2+2fou & -fou & \dots & 0 \\ . & . & . & . & . & . \\ . & . & . & . & . & . \\ . & . & . & . & . & . \\ 0 & 0 & 0 & \dots & -2fou & 2+2fou \end{bmatrix} \Bigg\} N-2$$

$$\{x\} = \begin{Bmatrix} u_1 \\ u_2 \\ \vdots \\ u_n \end{Bmatrix}$$

$$[B] = \begin{bmatrix} 2u_1^k + fou(u_{x_0}^k - 2u_{x_1}^k + u_{x_2}^k) \\ 2u_2^k + fou(u_{x_1}^k - 2u_{x_2}^k + u_{x_3}^k) \\ \vdots \\ 2u_{x_i}^k + fou(u_{x_{i-1}}^k - 2u_{x_i}^k + u_{x_{i+1}}^k) \end{bmatrix}$$

3. SAMPLE PROBLEM

$$BCs\ IC$$

$$u(0,t) = 0$$
$$u(L,t) = 0$$
$$u(x,0) = 100 \sin\left(\frac{\pi x}{0.8}\right)$$

Problem Variables,

L=0.8m

Rho=8920 kg/m^3

Cp=385.18544 J/kgC

k=397.48 W/mC

Analitic Solution

$$\alpha = \frac{k}{\rho c_p} = \frac{397.48}{8920 * 385.18544} = 0.0001157 \frac{W}{m^2 ℃}$$

$$c^2 = \alpha \ \Rightarrow c = \sqrt{\alpha} = 0.0107557$$

$$\lambda_1 = \frac{1 * c * \pi}{L} = \frac{0.0107557 * \pi}{0.8} = 0.0422377$$

$$B_n = \frac{2}{L}\int_0^L f(x) \sin\left(\frac{n\pi x}{L}\right) dx$$

$$f(x) = 100 \sin\left(\frac{\pi x}{0.8}\right)$$

$$B_1 = \frac{2}{0.8}\int_0^{0.8} 100 \sin\left(\frac{\pi x}{0.8}\right) \sin\left(\frac{1 * \pi * x}{0.8}\right) dx = 100$$

$$B_1 = 100, B_2 = B_3 = \cdots = 0$$

$$u(x,t) = \sum_{n=1}^{\infty} [B_n] \sin\left(\frac{n\pi x}{L}\right) e^{-\lambda_n^2 t}$$

Exact solution is

$$u(x,t) = 100 \sin\left(\frac{\pi * x}{0.8}\right) e^{-0.001784t}$$

Numerical Solution with Python

Explitic Method

```
import numpy as np
#Problem
L=0.8 #m
dx=0.1 #m
fou=0.1
t=1000 #s
k=397.48 #W/mC
rho=8920 #kg/m^3
cp=385.1854433 #J/kgC
#Values
nx = int((L/dx)+1) #number of points
alpha=k/(rho*cp) #thermal diffusivity
x_values=np.linspace(0,L,nx) #Points of temperature distrubition
dt = (fou*dx**2)/(alpha)
nt = int(t/dt) #time step
#Initial Condition of problem
u0=np.ones(nx) #Initial temperature distribution
for i in range(nx):
    u0[i]=100*np.sin((np.pi*x_values[i])/L)
#Boundry Conditions
Ts1=0
Ts2=0
u0[0]=Ts1
u0[-1]=Ts2

u=u0
#Function of explitic method
def explitic(fou,nx,T):
    for j in range(nt):
        Told = T.copy()
        for i in range(1, nx - 1):
            T[i] = Told[i] +  fou *(Told[i+1] - 2 * Told[i] + Told[i-1])
        T[0]=Ts1
        T[-1]=Ts2
    return (T)

expliticsol=explitic(fou,nx,u)
print(expliticsol)
```

Implitic Method

```
import numpy as np
#Problem
L=0.8 #m
dx=0.1 #m
fou=0.1
t=1000 #s
k=397.48 #W/mC
rho=8920 #kg/m^3
cp=385.1854433 #J/kgC
#Values
nx = int((L/dx)+1) #number of points
alpha=k/(rho*cp) #thermal diffusivity
x_values=np.linspace(0,L,nx) #Points of temperature distrubition
dt = (fou*dx**2)/(alpha)
nt = int(t/dt) #time step
#Initial Condition
u0=np.ones(nx) #Initial temperature distribution
for i in range(nx):
    u0[i]=100*np.sin((np.pi*x_values[i])/L)
#Boundry Conditions
u=u0
u0[0]=0.0
u0[-1]=0.0

def implitic(fou,T,nx):
    A=np.full((nx-2,nx-2),0.0)
    r=0
    for c in range(nx-2):
        if c==0 and r==0:
            A[r][c]=(1+2*fou)
            A[r][c+1]=-fou
            r=r+1
        elif c==nx-3 and r==nx-3:
            A[r][c-1]=-fou
            A[r][c]=(1+2*fou)
            r=r+1
        else:
            A[r][c-1]=-fou
            A[r][c]=(1+2*fou)
            A[r][c+1]=-fou
            r=r+1
    t_val=np.linalg.solve(A,T[1:-1])
    return t_val
tcounter=0
for j in range(nt):
    tcounter=tcounter+dt
    u[1:-1]=implitic(fou,u0,nx)
print(u)
```

Cranck-Nicolson Method

```
import numpy as np
L=0.8 #m
dx=0.1 #m
fou=0.1
t=1000 #s
k=397.48 #W/mC
rho=8920 #kg/m^3
cp=385.1854433 #J/kgC
nx = int((L/dx)+1) #number of points
alpha=k/(rho*cp) #thermal diffusivity
x_values=np.linspace(0,L,nx) #Points of temperature distrubition
dt = (fou*dx**2)/(alpha)
nt = int(t/dt) #time step
#Initial Condition and #Boundry Conditions
u0=np.ones(nx) #Initial temperature distribution
for i in range(nx):
    u0[i]=100*np.sin((np.pi*x_values[i])/L)
u=u0
u0[0]=0.0
u0[-1]=0.0
def CN(fou,T,nx):
    r=0
    A=np.full((nx-2,nx-2),0.0)#coefficient matrix
    for c in range(nx-2):
        if c==0 and r==0:
            A[r][c]=(2+2*fou)
            A[r][c+1]=-fou
            r=r+1
        elif c==nx-3 and r==nx-3:
            A[r][c-1]=-fou
            A[r][c]=(2+2*fou)
            r=r+1
        else:
            A[r][c-1]=-fou
            A[r][c]=(2+2*fou)
            A[r][c+1]=-fou
            r=r+1
    b=np.ones(nx-2)
    for  i in range(len(b)):
        if i==(len(b)-1):
            b[i]=2*T[i+1]+fou*(T[i]-2*T[i+1])
        else:
            b[i]=2*T[i+1]+fou*(T[i]-2*T[i+1]+T[i+2])
    t_val=np.linalg.solve(A,b)
    return t_val
for j in range(nt):
    u[1:-1]=CN(fou,u,nx)
print(u)
```

4. RESULTS, DISCUSSION and SUGGESTIONS

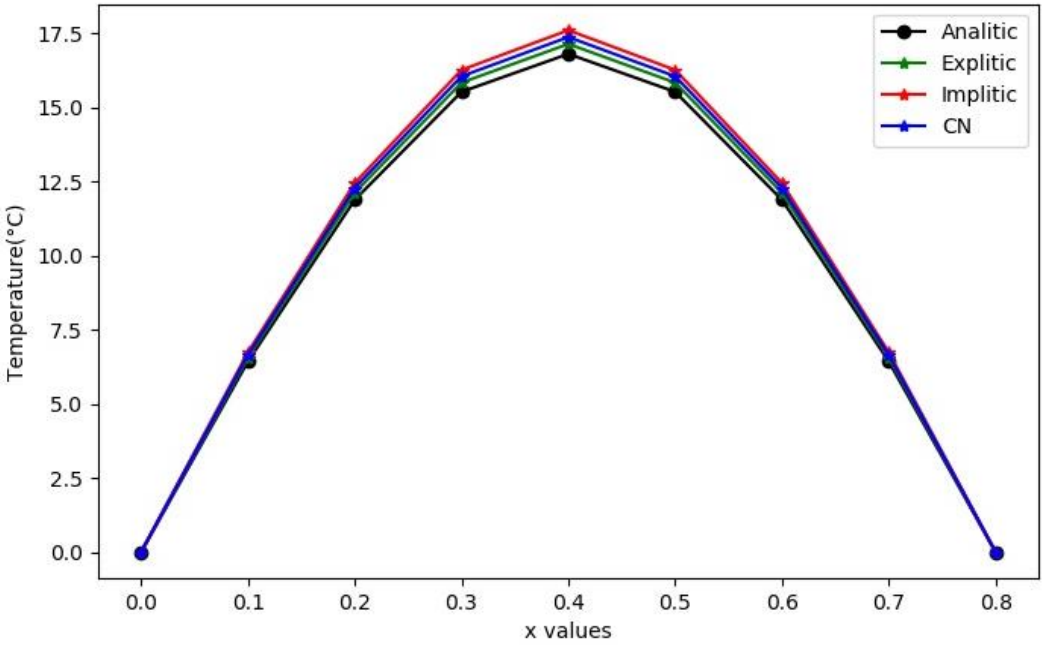
```
import matplotlib.pyplot as plt
import numpy as np

L=0.8
dx=0.1
nx=int((L/dx)+1)
analitic=np.array([0,6.427595,11.87664692,15.51758702,16.79611516,15.51758702,
11.87664692,6.427595,0])
explitic=np.array([0,6.55605128,12.11400318,15.82770791,17.13178759,15.8277079
1,12.11400318,6.55605128,0])
implitic=np.array([0,6.73316603,12.44126858,16.25530076,17.59461075,16.2553007
6,12.44126858,6.73316603,0])
cn=np.array([0,6.64469205,12.27779108,16.04170712,17.3634866,16.04170712,12.27
779108,6.64469205,0])
x=np.linspace(0,L,nx)
experror=np.ones(nx-1)
imperror=np.ones(nx-1)
cnerror=np.ones(nx-1)
#Error Calculations
for i in range(nx-1):
    experror[i]=(abs(analitic[i]-explitic[i])**2)
    imperror[i]=(abs(analitic[i]-implitic[i])**2)
    cnerror[i]=(abs(analitic[i]-cn[i])**2)
experr=np.sum(experror)/7
imperr=np.sum(imperror)/7
cnerr=np.sum(cnerror)/7

fig = plt.figure(figsize=(8,5))
axes= plt.axes()
axes.set_xticks([0,0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8])
ax = fig.add_subplot(111)
plt.xlabel("x values")
plt.ylabel("Temperature(°C)")
plt.plot(x,analitic,'-ok')
plt.plot(x,explitic,'-*g')
plt.plot(x,implitic,'-*r')
plt.plot(x,cn,'-*b')
plt.legend(["Analitic","Explitic","Implitic","CN"]) #plotting legend
print(experr)
print(imperr)
print(cnerr)
plt.show()
```

Temperature distribution at t=1000s. (ºC)

x values

	0	**0.1**	**0.2**	**0.3**	**0.4**	**0.5**	**0.6**	**0.7**	**0.8**
Analitic	0	6.427595	11.87664692	15.51758702	16.79611516	15.51758702	11.87664692	6.427595	0
Explitic	0	6.55605128	12.11400318	15.82770791	17.13178759	15.82770791	12.11400318	6.55605128	0
Implitic	0	6.73316603	12.44126858	16.25530076	17.59461075	16.25530076	12.44126858	6.73316603	0
CN	0	6.64469205	12.27779108	16.04170712	17.3634866	16.04170712	12.27779108	6.64469205	0

Table 4.1



Shape 4.1

Error Values

Methods	**RMSE**
Explitic	0.0643863
Implitic	0.3643401
Cranc-Nicolsen	0.1839157

Table 4.2

The method with the lowest error rate for the given boundary conditions in the calculations is the explicit method. Since the implementation of the Explicit method is faster than the other methods, it is appropriate to use the explicit method for the sample problem. The main reason for the differences in temperature profiles is the size of the fourier number. It is likely that the error rate will decrease if a smaller fourier number is used.

RESOURCES

[1] Kreyszig E., 12.6 Heat Equation: Solution by Fourier Series, Corliss S, Edwards Melissa, 10th Edition, JOHN WILEY & SONS, INC., United States of America, 558-561, 2011

[2] Recktenwald G., Portland State University, https://web.cecs.pdx.edu/~gerry/class/ME448/lecture/ME448_lecture_01.html (VisitingDate:13.06.2020)

[3] Recktenwald G., Portland State University, https://web.cecs.pdx.edu/~gerry/class/ME448/lecture/ME448_lecture_02.html (VisitingDate:13.06.2020)

[4] Recktenwald G., Portland State University, https://web.cecs.pdx.edu/~gerry/class/ME448/lecture/ME448_lecture_03.html (VisitingDate:13.06.2020)

[5] Recktenwald G., Portland State University, https://web.cecs.pdx.edu/~gerry/class/ME448/lecture/ME448_lecture_04.html (VisitingDate:13.06.2020)